

Build Exercise: A Chatbot That Remembers You

Time box: ~1 day. We mean it — if you're spending more, you're gold-plating something we didn't ask for. Use AI coding tools freely; we expect it.

The context

We're building a product whose core feature is *memory*: through ordinary conversation it learns durable things about each user and uses them in later chats — across separate sessions, the way a thoughtful person would. This exercise is a miniature of that core. We're not testing whether you can call an LLM API. We're testing whether you have judgment about what to remember, when memory should change, and how to get the right slice of it back in front of the model.

What to build

A command-line (or tiny web) chatbot with **per-user memory that persists across sessions**. A user can have a conversation, close the program, come back later in a fresh session, and the bot still knows what matters about them — and uses it.

The bot must:

1. Extract and store durable facts about a user from conversation.
2. Persist them outside the model's context (a file or a local DB is fine — no cloud infra needed).
3. Retrieve the relevant ones and use them when responding in a later session.
4. Keep each user's memory completely separate from every other user's.

The scenario it must handle

Your submission has to pass this exact script. Treat each session as a *separate run* of your program (the bot should not be relying on anything held in memory between runs except what it persisted). User is identified by a `(user_id)` you pass in however you like.

Session 1 — user `(maya)`

1. "Hey, I'm Maya. I'm a product manager at Stripe."
2. "Ugh, it's pouring rain here and I'm in a terrible mood lol."
3. "I'm training for a half marathon in October. Also I'm vegetarian."

Session 2 — user `(maya)` (new run) 4. "What do you know about me so far?" 5. "Quick update — I left Stripe, I'm a PM at Notion now." 6. "Suggest a post-run dinner for me."

Session 3 — user `(maya)` (new run) 7. "Where do I work again?"

Isolation check — user `(sam)` (brand new) 8. "Hi! What do you know about me?"

We're looking at specific moments in this: what the bot says it remembers at step 4, how step 5 affects storage, whether step 6 respects her constraints, what it answers at step 7, and what it says at step 8. You'll know which moments matter if you've built this before.

What we've deliberately left to you

These are your calls. We care more about *why* you chose than what you chose:

- When extraction runs and what it costs per message.
- What counts as worth remembering versus what doesn't.
- How memory is stored and structured.
- How you decide which memories to pull back for a given message.
- How you handle a fact that changes.

Constraints and non-goals

Any language. You may use libraries for the boring parts (embeddings, a vector store, a DB). **Implement the core remember-and-retrieve loop yourself** rather than handing the entire thing to an off-the-shelf memory framework — we want to see *your* decisions, not a library's defaults. If you'd use such a framework in production, say so in your write-up and tell us what it would handle.

Please **don't** spend the day on: authentication, a polished UI, multi-tenancy infra, deployment, or making it fast at scale. A clean CLI that works is perfect.

Deliverables

1. **Code that runs** with a one-command setup in the README ((how do I run the scenario above) should be obvious).
2. A **transcript or short screen recording** of the bot going through the full scenario.
3. **DECISIONS.md** — short, ~one paragraph each:
 - What do you treat as a memory versus ignore, and how did you decide?
 - How does extraction work — when does it run, what does it cost per message?
 - Walk us through exactly what happens in storage at step 5 (the job change).
 - How do you retrieve relevant memories for a given message, and why that approach at this scale?
 - If this ran for 100k users over a year, what breaks first and what would you change?

That last write-up matters as much as the code. We'd rather see a simple system with sharp reasoning than a clever one you can't explain.